

Germline Small-Variant Annotation and Pathogenicity Classification on NCHC (Hands-on)

A VEP-based annotation and point-based in-house ACMG classification workflow for germline small variants

Workflow developed by 李婕瑜 (Chieh-Yu Lee) • 袁自航 (Tzu-Hang Yuan) • Dr. 陳俞安 (Dr. Yu-An Chen)

Advisors: Dr. 陳沛隆 (Dr. Pei-Lung Chen) • Dr. 許書睿 (Dr. Jacob Shu-Jui Hsu) • Dr. 陳倩瑜 (Dr. Chien-Yu Chen)

Computation on NCHC • Draft 2026.

Germline Small-Variant Annotation and Pathogenicity Classification on NCHC (Hands-on)

Germline small-variant pathogenicity analysis · Hands-on guide. For users who need to rebuild this workflow on NCHC.

This guide covers three stages: ① VEP annotation → ② ACMG pathogenicity classification → ③ conditional filtering. The VEP output (`.vep.tsv`) is the input to ACMG, and the ACMG output (`.vep.ACMG.tsv`) is the input to filtering — run them in order.

What's new in this version ① `submit.sh` now supports `sample.list` submission. ② `filter_acmg.py` lives in `ACMG_filter/`, alongside `run_filter.sh` and `gene.list`. ③ The Conda environment installation path is flexible (either under `/home/$USER/` or `/work/$USER/`). ④ ACMG can be chained automatically after VEP completes (the `RUN_ACMG` switch). ⑤ Filtering has two modes — Classification (OR) and Pipeline (AND) — and the output always includes `ACMG_point` automatically.

0. Change these to your own:

Item	Example value	Replace with
Account ¹	d03455002	Your login account
Project code #SBATCH -A	MST109178	Your iService project ID
Notification email --mail-user	you@example.com	Your email

¹ If you are working in your account's `/work` directory, you do not need to replace `$USER` with your actual username in any of the subsequent steps.

Do NOT change any `/staging/reserve/jacobhsu/reference/VEP/...` paths (reference genome, VEP cache, plugins, custom annotations such as ClinVar/gnomAD, and the database paths in the ACMG `config.json`). These are shared lab data — if you have read access, use them directly. Verify first: `ls /staging/reserve/jacobhsu/reference/VEP/` should list contents.

1. Directory layout (final structure)

```
/work/$USER/VEP-ACMG/
├── VEP-small_variant/          # VEP annotation
│   ├── 00_vep_batch_submitter.py
│   ├── 01_preprocess.sh
│   ├── 02_submit_vep.sh
│   ├── 02_vep.sh
│   └── 03_post_vep.sh
├── ACMG-small_variant/        # ACMG classification
│   ├── acmg_batch_submitter.sh # Run ACMG classification standalone (Stage 5)
│   ├── sample_acmg.list       # Configuration required for standalone ACMG run
│   ├── run_acmg.sh
│   ├── main.py
│   └── modules/
│       ├── acmg_rules.py
│       ├── aggregation.py
│       └── data_loader.py
├── ACMG_filter/
│   ├── rescore.py             # Customize scores per rule and classification thresholds
│   ├── run_filter.sh          # Config-based launcher (Stage 6)
│   ├── filter_acmg.py         # Filtering script (Stage 6)
│   └── gene.list              # Gene list for --gene-list (empty by default)
├── utils/
│   ├── VEP/
│   ├── ACMG/
│   │   ├── config.json
│   │   └── hg38_BA1exc.tsv
│   ├── job_utils.sh
│   ├── environment.yml
│   └── setup_env.sh           # Interactive Conda environment installer (Stage 2)
├── input_vcf/                 # Your input VCF files
├── sample.list                 # Cases for VEP to process (customizable)
├── results/                   # All output files
│   ├── sample/
│   │   ├── VEP_output/
│   │   ├── ACMG_output/
│   │   ├── scripts/
│   │   ├── logs/             # Logs generated during the execution of the script for each individual sample
│   │   └── sample_submit.sh
│   └── logs/                 # Log generated when submitting the script
├── config.sh                  # Configuration file (Stage 3)
└── submit.sh                  # One-click execution script (Stage 4)
```

2. Stage 0 • Log in

```
ssh <account>@t3-c4.nchc.org.tw    # log in to NCHC
```

3. Stage 1 • Clone the github repo

```
cd /work/$USER
git clone https://github.com/Jacob-s-Lab/VEP-ACMG.git
```

4. Stage 2 • Set up environments

2-1 VEP: no installation needed. VEP uses the system module and the pre-built shared conda environment (`/opt/ohpc/Taiwania3/pkg/biology/vep/vep_v115`); you only need to load the module.

2-2 ACMG: create the conda environment acmg_rule.

Required only for the **first-time** setup.

```
cd /work/$USER/VEP-ACMG/utils
bash setup_env.sh
```

An interactive prompt will appear, allowing you to choose your Conda environment installation path based on your preference:

```

=====
Conda Environment Auto-Installer
=====
請選擇您的 Conda 環境安裝路徑：          # Please select your Conda environment installation path:
1) /home/user/.conda/envs/ (預設路徑)      # (Default path)
2) /work/user/.conda/envs/
3) 自訂路徑 (手動指定環境與快取路徑)      # Custom path (Manually specify environment and cache directory)
=====
請輸入選項 (1/2/3):                        # Please select an option (1/2/3):

```

Option 1 Installs the environment under your home directory following Conda's default behavior. **Option 2** Installs the environment under `/work/<account>/VEP-ACMG/utils/.venv/`, and package caching is directed to `/work/$USER/.conda/pkgs`.

5. Stage 3 • Configuration

```
cd /work/$USER/VEP-ACMG
```

Modify the custom settings block at the top of `config.sh`.

```

# Fill the mail address if you want to receive email notifications when the job ends or fails.
export USERMAIL=""

# Your working directory (i.e., where this script is located)
export WKDIR="/work/${USER}/VEP-ACMG"

# Automatically run ACMG after VEP completion
## 1 = annotation + pathogenicity classification
## 0 = Only annotation
export RUN_ACMG=1

```

If the `VEP-ACMG/` directory is created under `/work/$USER/`, no modification to `WKDIR` is required.

For details on `# Automatically run ACMG after VEP completion`, see Section 6.1.

6. Stage 4 • Run VEP annotation

Flow: put `*.vcf.gz` into `input_vcf/` → list the cases to run in `sample.list` → submit `submit.sh`. `submit.sh` picks only the cases in `sample.list` and automatically submits the three jobs: preprocess → VEP array → merge.

If you don't know how to edit the file or need to process all samples in `input_vcf/`, you can directly run `submit.sh` without editing `sample.list`. However, please make sure there are files in `input_vcf/`.

```

# 1) put input VCFs into input_vcf/
# 2) edit sample.list (one per line, lines starting with # are skipped)
# 3) submit
cd /work/$USER/VEP-ACMG
bash submit.sh
queue --me

```

sample.list format Each line can be ① a full filename (`xxx.vcf.gz`), ② a sample ID (`xxx`, auto-appends `.vcf.gz` / `.vcf`), or ③ a short-ID prefix (e.g. `A5034E`, matched against `input_vcf/A5034E*.vcf.gz`, excluding `.tbi` and `.gvcf.gz`). When a short ID is used, the output sample name is that short ID. A prefix matching more than one file triggers a warning — use a more specific name.

Any invalid entries or problem samples in `sample.list` will be logged in `results/skipped_samples.warnings`. If this file does not appear, it means all entries are valid.

After each sample finishes, `results/<sample>/VEP_output/` contains `<sample>.vep.tsv` (ACMG input) and `<sample>.vep.mane_plus_clinical.tsv`.

6.1 Auto-chain ACMG after VEP

After you submit `submit.sh`, once a sample's three VEP jobs finish, Slurm dependency automatically submits that sample's ACMG classification — you no longer need to run `acmg_batch_submitter.sh` manually.

How to set `RUN_ACMG`: the switch is at the top of `submit.sh`, in the "customize settings" block. The default is already `1` (auto-chain ACMG after VEP); set it to `0` if you only want to run VEP and do Stage 5 yourself later. Save the file and submit as usual with `bash submit.sh`.

7. Stage 5 • Run ACMG classification (manual/batch, optional)

Users who enabled auto-chaining can skip this section; it is for `RUN_ACMG=0`, re-running, or re-processing all samples at once.

5-1 Set the input directory. List the absolute paths of all files to be processed by ACMG classification in `sample_acmg.list`.

If you don't know how to edit the file or need to process all samples in `results/`, you can directly execute `ACMG-small_variant/acmg_batch_submitter.sh` without editing `sample_acmg.list`.

Any invalid entries or problem samples in `sample_acmg.list` will be logged in `results/acmg_skipped_samples.warnings`. If this file does not appear, it means all entries are valid.

5-2 Submit manually:

```
cd /work/$USER/VEP-ACMG/ACMG-small_variant
bash acmg_batch_submitter.sh
queue --me
```

Results are in `results/<sample>/ACMG_output/`: `<sample>.vep.ACMG.tsv` and `<sample>.vep.mane_plus_clinical.ACMG.tsv` (containing `Pathogenicity_class` and `ACMG_rules`).

8. Stage 6 • Conditional filtering (filter_acmg.py)

Located in `ACMG_filter/`. It has two mutually exclusive modes (auto-detected), and the output always adds the `ACMG_point` column automatically. Use `-h` to see the options (`-h` works without activating the environment); `--list-columns` lists all columns.

- **Classification mode:** using any of `--plp` / `--clinVar-plp` / `--dvd-plp` → the result is their OR union, and all pipeline conditions below are ignored.
- **Pipeline mode:** only when no classification flag is used does it apply AND filtering in a fixed order.

8.1 Method A: config-style launcher run_filter.sh (recommended)

The `run_filter.sh` in the same folder wraps environment activation and path handling for you — edit "2) Filtering conditions", simply remove the leading `#` from the lines corresponding to the conditions you want to enable. When executing, append the target filename (e.g., `A5034E.vep.ACMG.tsv`).

```
cd /work/$USER/VEP-ACMG/ACMG_filter
bash run_filter.sh <filename> # You can also submit to Slurm via: sbatch run_filter.sh <filename>
```

sbatch path note The script automatically looks for the file under `results/<sample_id>/ACMG_output/` based on the provided filename, so full paths are not required. **Log location** All logs are written to `results/logs/`.

8.2 Method B: call filter_acmg.py directly

```
cd /work/$USER/VEP-ACMG/ACMG_filter
source ../config.sh
activate_conda_env
python filter_acmg.py -i <input>.vep.ACMG.tsv -o <out>.tsv [options]
```

8.3 Options (1) Classification mode (standalone; OR union; ignores pipeline conditions)

Option	Effect
<code>--plp</code>	ACMG <code>Pathogenicity_class</code> = Pathogenic / Likely_pathogenic
<code>--clinVar-plp</code>	<code>ClinVar_CLNSIG</code> = Pathogenic / Likely_pathogenic (the counterpart of <code>--ex-blb</code>)
<code>--dvd-plp</code>	DVD classification = Pathogenic / Likely_pathogenic

8.4 Options (2) Pipeline mode (AND; applied in this fixed order)

#	Option	Effect
1	<code>--pass-only</code>	Keep only FILTER = PASS
2	<code>--min-dp INT</code>	Keep DP ≥ value (nulls removed)
3	<code>--acmg-ex RULE[,RULE]</code>	Remove rows whose <code>ACMG_rules</code> contain the exact token(s) (e.g. BA1)

#	Option	Effect
—	<code>--ex-blb</code>	Remove ClinVar pure-benign (has B/LB and no conflicting/VUS/P)
4	<code>--min-vaf FLOAT</code>	Keep VAF $\geq$ value (nulls removed)
—	<code>--max-af FLOAT</code>	Both genome_AF and exome_AF $\leq$ value (nulls kept); 0.01 / 0.05
5	<code>--gene-list PATH</code>	Keep only sites whose SYMBOL is in the gene list (one per line)
6	<code>--exonic / --noncoding</code>	Keep only class-1 exonic+splice / keep only non-class-1 (choose one)
7	<code>--consequence + --mode</code>	Custom Consequence keywords; --mode include/exclude
8	<code>--acmg-ex-b-rule-only</code>	Remove rows with only B evidence and no P evidence
9	<code>--vushigh</code>	Keep only <code>ACMG_point</code> total ≥ 3 (VUS-high)

Shared: `--list-columns` / `-v` (list columns / show remaining row counts per step). The order in which you pass options does not affect the result (an AND intersection is order-independent; only the intermediate `-v` counts follow the fixed order).

8.5 ACMG point rules (ACMG_point is always computed automatically)

P-prefixed criteria are positive, B-prefixed negative; the `_modifier` (`_VERYSTRONG` / `_STRONG` / `_MODERATE` / `_SUPPORTING`) overrides the strength. `--vushigh` keeps only totals ≥ 3 .

Strength	Pathogenic (+)	Benign (–)
very strong	PVS = +8	BA = –8
strong	PS = +4	BS = –4
moderate	PM = +2	BM = –2
supporting	PP = +1	BP = –1

Examples: `PVS1,PM2 = 8+2 = 10`; `PVS1_STRONG = +4`; `BP6_STRONG = –4`; `BA1 = –8`.

8.6 Examples

```
# Classification mode: keep the union of ACMG-P/LP or DVD-P/LP (score auto-included)
python filter_acmg.py -i MDD261-PKD.vep.ACMG.tsv -o class.tsv --plp --dvd-plp -v

# Pipeline mode: PASS + drop BA1 + coding + VUS-high (AND)
python filter_acmg.py -i MDD261-PKD.vep.ACMG.tsv -o flow.tsv \
  --pass-only --acmg-ex BA1 --exonic --vushigh -v
```

9. Compute resources (SBATCH settings per script)

Script	Partition	Cores	Memory	Notes
01_preprocess.sh	ngs13G	2	13 GB	VCF normalization, split by chromosome
02_submit_vep.sh	ngs7G	1	7 GB	Control / submit array
02_vep.sh	ngs92G	14	92 GB	VEP annotation (array)
03_post_vep.sh	ngs7G	1	7 GB	Merge, produce TSV
run_acmg.sh	ngs26G	4	26 GB	ACMG classification (array)
run_filter.sh	ngs7G	1	7 GB	Conditional filtering (optional sbatch)